

Namaste Python

Visual & Easy-to-Understand Revision Guide

The goal is not to memorize syntax. Understand the mental model first, then use the code as a reminder.



NAMASTE PYTHON

Concept-First Revision Guide

Understand • Remember • Apply • Interview Ready

Python in One Line

Python is a high-level, interpreted, general-purpose programming language with simple syntax and powerful libraries.

ROADMAP

- 1 Python Basics
- 2 Data Types & Variables
- 3 Operators
- 4 Control Flow
- 5 Functions
- 6 *args & **kwargs
- 7 Scope & LEGB Rule
- 8 Closures & Decorators
- 9 Iterators & Generators
- 10 OOP (Object Oriented Programming)
- 11 Exceptions
- 12 Context Managers
- 13 Modules & Packages
- 14 Python Internals
- 15 Async Python
- 16 FastAPI Basics
- 17 Pydantic
- 18 SQLAlchemy Basics
- 19 Testing with pytest
- 20 Logging
- 21 Interview Q & A
- 22 Rapid Revision Sheet

WHY PYTHON?

- ✓ Easy to learn
- ✓ Readable & clean syntax
- ✓ Huge standard library
- ✓ Cross-platform
- ✓ Great for Web, Data, AI/ML, Automation & more

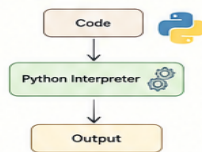
TIP

Focus more on concepts than syntax. Understand 'why' something works, not just 'how'.



1. PYTHON BASICS

- Python code is executed line by line.
- Indentation (spaces) is mandatory.
- Everything is an object in Python.



5. CONTROL FLOW

```
if / elif / else
x = 10
if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
for loop
for i in range(3):
    print(i) # 0 1 2
while loop
i = 0
while i < 3:
    print(i)
    i += 1
```

8. SCOPE & LEGB RULE

Python looks for a variable in this order:

- L** Local (inside current function)
- E** Enclosing (inside enclosing function)
- G** Global (in global space)
- B** Built-in (python built-in names)

If not found at any level, NameError is raised.

11. OOP IN PYTHON

```
Class & Object
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return f"Hi, I am {self.name}"
p = Person("Alice")
print(p.greet())
```

Pillars of OOP

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

2. VARIABLES & DATA TYPES

```
x = 10 # int
pi = 3.14 # float
name = "Alice" # str
is_valid = True # bool
items = [1, 2, 3] # list
person = {"name": "Bob"} # dict
unique = {1, 2, 3} # set
point = (1, 2) # tuple
```

int	float	str	bool
10	3.14	"Hi"	True
list	tuple	set	dict
[1,2,3]	(1,2)	{1,2,3}	{'a':1}

6. FUNCTIONS

```
def greet(name):
    """This is a docstring"""
    return f"Hello, {name}!"
print(greet("Alice"))
```

Types of Functions

1. Built-in (len(), print(), type())
2. User-defined
3. Lambda (anonymous)

Lambda Example

```
square = lambda x: x ** 2
print(square(4)) # 16
```

9. CLOSURES & DECORATORS

Closure Example

```
def outer(msg):
    def inner():
        print(msg)
    return inner
hello = outer("Hi")
hello() # Hi
```

Decorator Example

```
def decor(func):
    def wrapper():
        print("Before")
        func()
        print("After")
    return wrapper
@decor
def say_hello():
    print("Hello")
say_hello()
```

12. EXCEPTIONS

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print("Cannot divide by zero")
else:
    print("No Exception")
finally:
    print("This will always run")
```

Common Exceptions

- ZeroDivisionError
- ValueError
- TypeError
- FileNotFoundError
- KeyError

15. GIL (GLOBAL INTERPRETER LOCK)

- Allows only one thread to execute Python bytecode at a time.
- Good for I/O-bound tasks (not CPU-bound).
- Multiprocessing is used for CPU-bound tasks.



3. MUTABLE vs IMMUTABLE

Mutable
(Can be changed)

- list
- dict
- set
- bytearray



Immutable
(Cannot be changed)

- int
- float
- str
- tuple
- bool
- frozenset



4. == vs is

== (Value comparison)
`a = [1, 2, 3]`
`b = [1, 2, 3]`
`print(a == b) # True`

is (Identity comparison)
`a = [1, 2, 3]`
`b = a`
`print(a is b) # True`

Use == to compare values
Use is to check same object in memory

7. *args & **kwargs

***args** (Non-keyword arguments)

```
def add(*numbers):
    return sum(numbers)
print(add(1, 2, 3, 4)) # 10
```

****kwargs** (Keyword arguments)

```
def info(**data):
    for k, v in data.items():
        print(k, "=", v)
info(name="Alice", age=25)
```

10. ITERATORS & GENERATORS

Iterator

```
it = iter([1,2,3])
print(next(it)) # 1
print(next(it)) # 2
print(next(it)) # 3
```

Generator (using yield)

```
def count(n):
    i = 1
    while i <= n:
        yield i
        i += 1
for num in count(3):
    print(num)
```

Generators are memory efficient (lazy evaluation).

13. CONTEXT MANAGERS

```
with open("file.txt", "r") as f:
    data = f.read()
# file is automatically closed
```

Custom Context Manager

```
class MyContext:
    def __enter__(self):
        print("Enter")
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exit")
with MyContext():
    print("Inside block")
```

14. PYTHON MEMORY MODEL (HIGH LEVEL)

Stack (Primitive)
int, float, bool

Heap (Objects)
list, dict, class, etc.

Objects in heap are referenced by variables in stack.



16. ASYNC IN PYTHON

```
import asyncio
async def main():
    print("Start")
    await asyncio.sleep(1)
    print("End")
asyncio.run(main())
```



★ REMEMBER: Practice code daily • Build projects • Solve problems • Revise regularly → You will master Python! 🚀

1. How Python Thinks

In simple words: Python variables are names pointing to objects. The object lives in memory; the variable gives you a way to reach it.

Mental model: name → object → value/type. Assignment usually changes what a name points to.

See it in code

```
a = [1, 2]
b = a
b.append(3)
print(a) # [1, 2, 3]
```

Remember: When two names point to the same mutable object, changing it through one name is visible through the other.

Interview question: What is the difference between a variable, an object, and a reference?

2. Mutable vs Immutable

In simple words: Mutable objects can change in place. Immutable objects cannot; a new object is created when you appear to change the value.

Mental model: list/dict/set → usually mutable. int/float/str/tuple/bool → immutable.

See it in code

```
items = [1, 2]
items.append(3) # mutation

name = 'Sam'
name = name + ' Hazra' # rebinding
```

Remember: Mutation changes an existing object. Rebinding makes a name point somewhere else.

Interview question: Why can a tuple contain a mutable list even though the tuple itself is immutable?

3. == vs is

In simple words: `==` asks: do these objects have equal values? `is` asks: are these the exact same object?

Mental model: Think: value comparison vs identity comparison.

See it in code

```
a = [1, 2]
b = [1, 2]
print(a == b) # True
print(a is b) # False
```

Remember: Use `is` for identity checks such as `value is None`.

Interview question: Why should you normally use `is None` instead of `== None`?

4. Functions → Scope → Closure → Decorator

In simple words: These concepts build on each other. A function can access names from enclosing scopes; a closure remembers them; a decorator uses this ability to wrap functions.

Mental model: Function → nested function → captured state → wrapper.

See it in code

```
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = multiplier(2)
print(double(5)) # 10
```

Remember: A closure remembers state from the enclosing scope.

Interview question: What is a closure, and how is it related to a decorator?

5. Iterable → Iterator → Generator

In simple words: An iterable is something you can loop over. An iterator produces one item at a time. A generator is an easy way to create an iterator using `yield`.

Mental model: for loop → iter() → next() → StopIteration.

See it in code

```
def numbers():  
    yield 1  
    yield 2  
    yield 3  
  
for n in numbers():  
    print(n)
```

Remember: Generators are lazy: values are produced only when needed, which is useful for large data.

Interview question: What is the difference between an iterable, an iterator, and a generator?

6. OOP: Four Core Ideas

In simple words: Encapsulation groups data and behavior. Inheritance reuses/extends behavior. Polymorphism lets different objects respond to the same interface.

Mental model: Class = blueprint. Object = instance. Method = behavior attached to an object/class.

See it in code

```
class User:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f'Hi {self.name}'
```

Remember: Know ``self``, ``__init__``, instance vs class attributes, inheritance, MRO, and ``super()``.

Interview question: Explain instance method vs classmethod vs staticmethod.

7. Exceptions

In simple words: Exceptions are Python's way to signal that normal execution cannot continue as expected.

Mental model: try → exception? → except → optional else → finally cleanup.

See it in code

```
try:
    value = int('abc')
except ValueError:
    value = 0
finally:
    print('cleanup')
```

Remember: Catch specific exceptions. Do not hide errors with a broad bare ``except``.

Interview question: What is the difference between ``except``, ``else``, and ``finally``?

8. Context Managers

In simple words: A context manager makes resource handling safe and automatic.

Mental model: enter resource → use resource → guaranteed cleanup.

See it in code

```
with open('data.txt') as file:
    data = file.read()
# file is closed automatically
```

Remember: Think ``with`` whenever you acquire something that must be released.

Interview question: How do ``__enter__`` and ``__exit__`` work?

9. Memory + GIL

In simple words: Python objects live in memory and names reference them. CPython manages memory with reference counting plus cyclic garbage collection.

Mental model: name → object in heap. Standard CPython's GIL historically limits concurrent execution of Python bytecode within one interpreter.

See it in code

```
a = []
b = a
# two names reference one object

del a
```

```
# b still references the object
```

Remember: Threading is commonly useful for I/O-bound work; multiprocessing is useful for CPU-bound parallel work. Async is another model for I/O concurrency.

Interview question: What is the GIL, and why doesn't it mean Python cannot do concurrency?

10. Async Python: The Big Picture

In simple words: Async Python lets one thread make progress on other tasks while an I/O operation is waiting.

Mental model: coroutine → event loop → await I/O → task pauses → another task runs → resume.

See it in code

```
async def fetch():
    data = await get_data()
    return data

results = await asyncio.gather(fetch(), fetch())
```

Remember: Do not put long blocking operations inside an async path unless you deliberately move them off the event loop.

Interview question: What exactly happens when Python reaches `await`?

11. FastAPI Mental Model

In simple words: A request enters the route, dependencies are resolved, input is validated, business logic runs, and the response is serialized.

Mental model: HTTP request → route → Depends → Pydantic validation → service → DB → response.

See it in code

```
from fastapi import Depends

@app.get('/me')
def me(user=Depends(get_current_user)):
    return user
```

Remember: Keep authentication, DB sessions, and reusable request logic in dependencies/services rather than duplicating them.

Interview question: Why is FastAPI dependency injection useful?

12. SQLAlchemy Mental Model

In simple words: The Session is the unit that tracks ORM objects and manages transactions. Querying, flushing, committing, and rolling back are different operations.

Mental model: Python object → Session → SQL → database. flush sends SQL; commit finalizes the transaction.

See it in code

```
user = User(name='Sam')
session.add(user)
session.flush() # SQL sent
session.commit() # transaction committed
```

Remember: Learn N+1, lazy loading, joinedload/selectinload, transactions, and async sessions.

Interview question: What is the difference between flush and commit?

13. Testing

In simple words: A good test proves behavior and catches regressions without depending too heavily on implementation details.

Mental model: Arrange → Act → Assert.

See it in code

```
def test_add():
    result = 2 + 3
    assert result == 5
```

Remember: For FastAPI, know fixtures, dependency overrides, test clients, mocking, and database isolation.

Interview question: How would you test a FastAPI endpoint that depends on the current user?

14. Your Python Interview Map

In simple words: For interviews, move from fundamentals to internals and then to backend-specific design.

Mental model: Python basics → functions/scope → iterators/generators → OOP → memory/GIL → async → FastAPI → SQLAlchemy → testing.

Remember: If you can explain the mental model without looking at code, you are ready to revise the syntax.

Interview question: Be prepared to explain not just what something does, but why it behaves that way.

15. One-Page Final Revision

Topic	Remember this
Variables	Names point to objects
Mutable	Can change in place
Immutable	Cannot change in place
==	Value equality
is	Object identity
LEGB	Local → Enclosing → Global → Built-in
Closure	Inner function remembers enclosing state
Decorator	Wrap/extend callable behavior
Iterable	Can produce an iterator
Iterator	Produces next item + maintains state
Generator	Lazy iterator using yield
OOP	Class, object, inheritance, polymorphism
MRO	Method lookup order
Context manager	Safe setup + cleanup
GIL	CPython bytecode execution constraint
async/await	Cooperative I/O concurrency
Event loop	Schedules/resumes async tasks
FastAPI Depends	Reusable dependency graph
Pydantic	Validation + serialization
SQLAlchemy Session	ORM identity + transaction management
flush	Send pending SQL within transaction
commit	Finalize transaction
pytest	Arrange → Act → Assert

Revision rule: Read the visual → explain it aloud → write one example from memory → solve one interview question.